

Lesson Notes

MODULE 7 | LESSON 2

THE CORRELATION MATRIX RELOADED

Reading Time	60 minutes
Prior Knowledge	Traditional portfolio theory and utility theory.
Keywords	behavioral finance, bounded rationality, satisfice, prospect theory, cognitive biases, emotional biases.

The last lesson provided techniques to improve the covariance matrix estimate, specifically making it more robust generally in order to get better performing portfolios. In this lesson, we continue to improve our estimate of the covariance matrix. The efficient portfolio optimization introduced by Pedersen et al. specifically tackles the “problem portfolios” with its shrinkage approach for the correlation matrix (not the covariance matrix). De Prado, on the other hand, focuses on minimizing or removing the aspects of our covariance estimate that do not help us construct better portfolios. Specifically, we minimize the impact of what we consider noise and we remove the impact of the overall market factor.

1. Problem Portfolios

In the last lesson, we discussed the instability problem of mean-variance optimization (MVO), that is, how even very small errors in the estimation of the covariance matrix have a large impact on the output of the MVO, the optimal portfolio allocation. The first reading for this lesson provides more details about the consequences of misspecifying the covariance matrix input. Pedersen et al. provide a very straightforward description of the mechanism by which this misspecification degrades portfolio performance by framing it in terms of principal components (PCs). They note that MVO allocates to each PC based on its excess return divided by volatility, or Sharpe ratio. They also observe that the smallest PCs (those with the least volatility) are the most susceptible to two types of error that are disastrous for MVO: The volatilities are underestimated and the returns are overestimated. This implies that the Sharpe ratio will be drastically overestimated due to effects in both the numerator and the denominator. MVO will then overallocate to these PCs due to their very high apparent Sharpe ratio, and the optimal portfolio will be doomed to underperform expectations.

2. Efficient Portfolio Optimization (EPO)

Pedersen’s solution to this problem “is to reduce the estimated Sharpe ratios of the least important principal components,” which surprisingly they accomplish by shrinking the original assets’ correlations via a very simple convex combination of the correlation matrix with the identity matrix (3–4). This shrunken correlation matrix is then used to formulate the covariance matrix used as an input to MVO. This reformulation of the covariance matrix is discussed in more detail at the end of these lesson notes.

Note that this operation is similar to the convex combination involved in the shrinkage techniques from the last lesson, except that we are dealing here with the correlation matrix instead of the covariance matrix. Still, similar to the last lesson, we need to determine the shrinkage coefficient or EPO shrinkage parameter. In other words, we need to determine how much to shrink, which is itself an optimization problem. On the one hand, the shrinkage parameter should provide the MVO problem with enough information from the original correlation matrix that the resulting portfolio can take maximum advantage of diversification potential; on the other hand, the shrinkage should be sufficient to avoid the instability issues discussed in the last lesson and the inefficient portfolio allocation mentioned above. Beyond that, Pedersen’s shrinkage technique lends itself to optimization of the EPO shrinkage coefficient with respect to any of the following approaches to portfolio modeling:

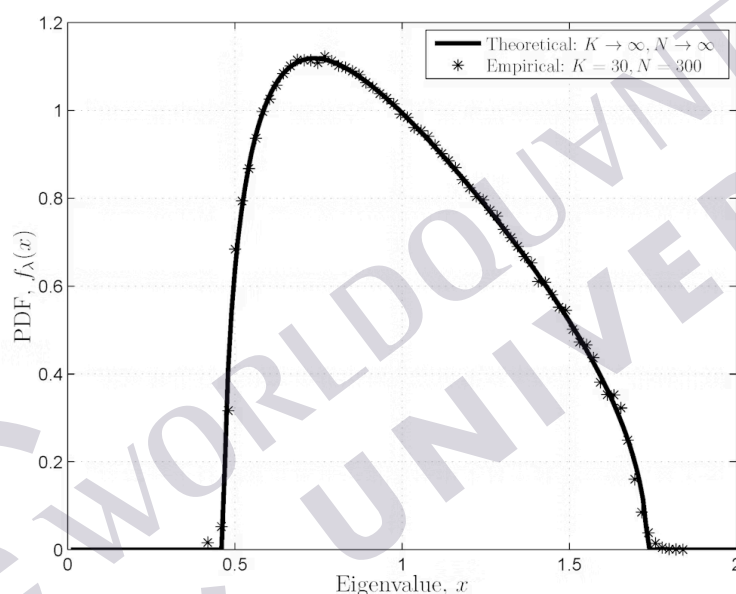
1. A Bayesian approach that incorporates prior conviction about asset returns as you have seen in the Black-Litterman optimization;
2. An "anchor portfolio" for the case where a portfolio manager wants to improve performance without straying too far from the benchmark portfolio to which performance will be compared;
3. Robust optimization, an alternative to the robust estimation technique that was mentioned in the last lesson, which directly incorporates the returns estimation error by way of uncertainty parameters—in addition to the usual variance of returns that we expect to feed the MVO problem.
4. Ridge regression regularization, the machine learning tool with which you are by now very familiar.

Pedersen provides a relatively straightforward theoretically accessible proof for the unification of all of these, so pay attention to this mathematical development.

3. De-Noising

De Prado provides yet another approach to modifying the covariance matrix input to the MVO that also relies on an understanding of the principal components analysis of the covariance matrix. But first, a quick foray into random matrix theory, according to which the eigenvalues of a completely random matrix are probabilistically distributed according to the Marcenko-Pastur distribution (MPD) as shown below.

Figure 1: Theoretical and Empirical Marcenko-Pastur Distribution

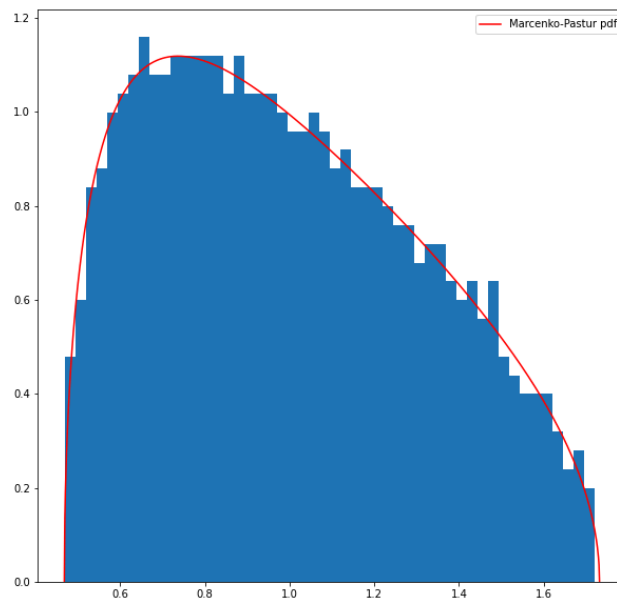


Source: Zhang, Wensheng, Cheng-Xiang Wang, Xiaofeng Tao, and Piya Patcharamaneepakorn. "Exact Distributions of Finite Random Matrices and Their Applications to Spectrum Sensing." *Sensors*, vol.16, no. 8, 2016, <https://doi.org/10.3390/s16081183>.

The guiding insight of de Prado's de-noising technique is that, since we know the distribution of the eigenvalues of a completely random (i.e., noisy) matrix, then any eigenvalues of the covariance matrix that are larger than we expect based on this distribution probably relate to signal, or meaningful information, as opposed to noise. Therefore, by reducing the impact of the noisy PCs (those with eigenvalues that fit to MPD) in our MVO, the result should be a better portfolio—a portfolio constructed using less noise or error.

To better see and understand de Prado point, let's take a random matrix of size 1,000 x 10,000, and display the eigenvalues of this random matrix as a histogram (in blue), and then overlay the probability density function (PDF) for the MPD (red line):

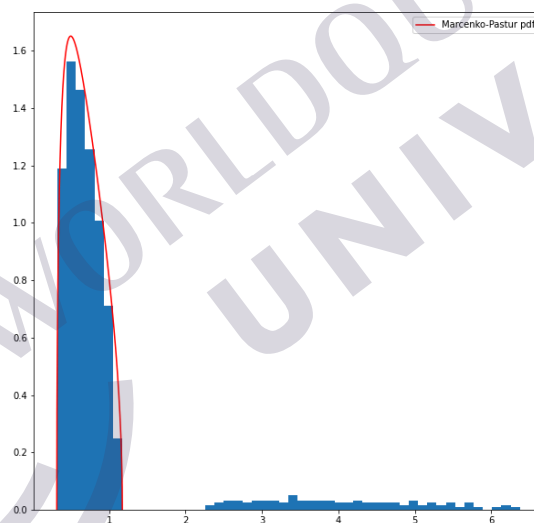
Figure 2: Histogram of Noisy Eigenvalues Fitted to Marcenko-Pastur Distribution



As you can see, we get a pretty good fit between the eigenvalues of the completely random matrix and the PDF for the MPD, as we expected based on random matrix theory.

Now, instead of a completely random matrix, i.e., one consisting of only noise, let us assume that there are some non-random variables, i.e., signal. You can use the code from Moen's Github repo to reproduce this example. In this case, we have inserted 100 signal factors into the random matrix constructed above. Let us plot the histogram of the new eigenvalues as well as the PDF for the MPD as before:

Figure 3: Histogram of Noisy and Signal Eigenvalues Fitted to Marcenko-Pastur Distribution



As we can see, there are many eigenvalues to the right of the (red) PDF line. Again, since the red line roughly captures the distribution of the eigenvalues, which we would expect from a completely random matrix, the eigenvalues some distance to the right are probably not random. The eigenvectors associated with these 100 non-random eigenvalues should be considered signal and used to construct the optimal portfolio.

Check out these two videos that discuss fitting signal and noise to the MPD to gain some more intuition about how this works.

06:43

[Access video transcript here](#)

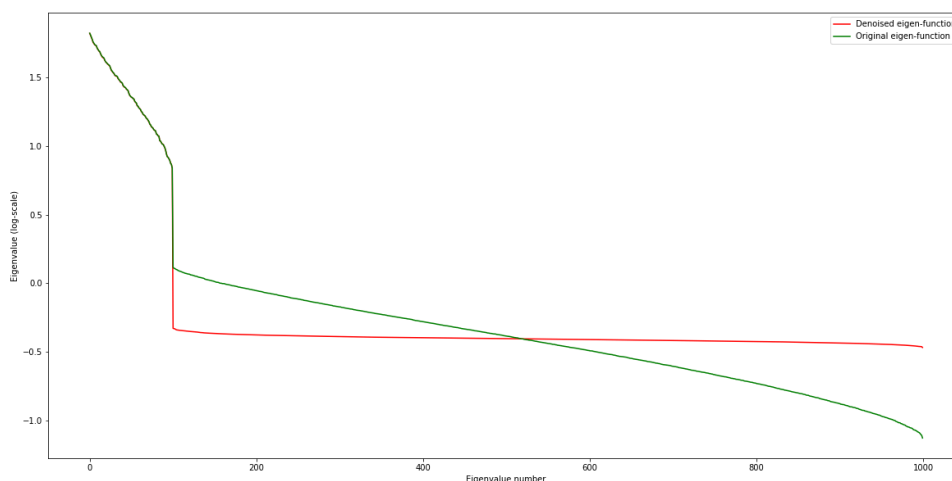
11:26

[Access video transcript here](#)

De Prado provides two ways to reduce the impact of the noisy PCs, “Constant Residual Eigenvalue Method” (CREM) and “Targeted Shrinkage.” The latter incorporates the equivalent of a shrinkage coefficient, allowing the portfolio manager to specify or regulate the degree of de-noising. CREM, on the other hand, is a more holistic approach: After performing the eigenvector decomposition of the correlation matrix, *all* of the noisy eigenvalues in the diagonal matrix are replaced by their average. This reduces the largest of these noisy eigenvalues and preserves the trace (the sum of diagonal elements, i.e., the eigenvalues). Obviously, the smallest of the noisy eigenvalues are increased, but their values are still small compared to the eigenvalues of the principal components associated with signal.

We use the same random matrix with 100 non-random factors added. Notice that in the red line representing the de-noised eigenvalues, all but the first 100 eigenvalues are flattened out by the averaging:

Figure 4: Eigenvalues Before and After De-Noising



The Python code to execute this de-noising is straightforward, requiring only the array of eigenvalues (`eVal`), the array of eigenvectors (`eVec`), and the number of PCs that we consider to be non-random (`nFacts`):

```
def denoisedCorr(eVal, eVec, nFacts):

    eVal_ = np.diag(eVal).copy()

    eVal_[nFacts:] = eVal_[nFacts:].sum()/float(eVal_.shape[0] - nFacts)

    eVal_ = np.diag(eVal_)

    corr1 = np.dot(eVec, eVal_).dot(eVec.T)

    return corr1
```

The first two inputs are easily obtained starting with the function `numpy.linalg.eig()`, but even better is to use the function `getPCA()` from Moen's snippet2.2, which returns them in the proper order—that is, so that the eigenvalues are in descending value as you move down the diagonal of their matrix, and the eigenvectors are in the same order as their corresponding eigenvalues. For the third input, you first need to find the largest eigenvalue you would expect to find according to the MPD for the given matrix size, which is considered the cut-off for randomness; as discussed, any eigenvalues larger than this are considered non-random or signal, which we want our MVO to exploit. However, many eigenvalues are larger than the MPD would predict is the scalar input provided as `nFacts`.

The main steps to perform CREM de-noising are as follows:

1. Use `getPCA` from snippet 2.2 to get the eigenvectors and eigenvalues.
2. Use `findMaxEval()` from snippet 2.4 to find the largest noisy eigenvalue we would expect from a completely random matrix the size of the correlation matrix. Any eigenvalues larger than this are considered non-random, and the number of such eigenvalues is `nFacts`. **Note:** In order to fit the MPD, you need the quotient q , where $q = T/N$ and N is the number of columns or assets and T is the number of rows or data points for these.
3. The denoised correlation matrix, along with the eigenvector matrix `std`, is an input to the function `corr2cov()`, which recomposes the covariance matrix.

With that said, you can simply use the function `deNoiseCov`. It takes the empirical covariance matrix, the aforementioned q , and a bandwidth parameter (`bwidth`) as arguments. Bandwidth is a smoothing parameter used in the kernel density estimation (KDE); it serves to regulate the familiar trade-off between variance and bias in the KDE. KDE helps estimate the probability density function (PDF) for unknown distributions. While the function in Moen's github `findOptimalBWidth`

performs a grid search to optimize the hyperparameter for you (using leave-one-out as the cross-validation approach), the additional required readings about KDE and specifically the importance of the bandwidth parameter should complete your understanding of this important non-parametric PDF estimation procedure. De Prado does not explain his default choice of the Gaussian kernel, but when you compare the MDP to the six kernel choices from the scikit-learn library, you may agree that it was an intuitive choice in that they are both smooth, and the other two smooth kernel choices are much more peaked than the MCD which we are trying to approximate.

4. De-Toning

De Prado refers to his other recommended technique for modifying the covariance matrix input to MVO as “de-toning” because it “is similar to removing a loud tone that prevents us from hearing other sounds” (31). In this case, the “loud tone” is the overall market factor similar to what the capital asset pricing model (CAPM) refers to as the market beta. Since all assets are affected by the rise and fall of the overall market—and according to CAPM, this risk cannot be diversified away anyway—this factor or tone should be removed from the covariance matrix before MVO. Removing the market factor (i.e., de-toning) effectively consists of subtracting the “market” correlation matrix (see below `corr_mark`, the matrix dot product) from the de-noised correlation matrix. This de-toned, de-noised correlation matrix can then be used to formulate the de-toned, de-noised covariance matrix for input to the MVO.

```
def detoned_corr(corr, eigenvalues, eigenvectors, market_component=1):

    # Getting the eigenvalues and eigenvectors related to market component

    eigenvalues_mark = eigenvalues[:market_component, :market_component]

    eigenvectors_mark = eigenvectors[:, :market_component]

    # Calculating the market component correlation

    corr_mark = np.dot(eigenvectors_mark, eigenvalues_mark).dot(eigenvectors_mark.T)

    # Removing the market component from the de-noised correlation matrix

    corr = corr - corr_mark

    # Rescaling the correlation matrix to have 1s on the main diagonal

    corr = cov2corr(corr)

    return corr
```

In these situations where we modify the correlation matrix, we can reformulate the covariance matrix by reversing the eigendecomposition process: Left-multiplying the correlation matrix by the eigenvector matrix (`std`) and right-multiplying this product with the inverse of the eigenvector matrix, or simply using the `numpy.outer()` method as deployed in this function from code snippet 2.3 (Moen):

```
def corr2cov(corr, std):

    cov = corr \times np.outer(std, std)

    return cov
```

This linear algebra function can also be used to reformulate the covariance matrix from the correlation matrix that results from the EPO described in the first part of this lesson.

5. Conclusion

With Pedersen's EPO and de Prado's de-noising and de-toning, we now have two additional techniques to improve the covariance matrix (by modifying the correlation matrix embedded in it) before input to MVO. The goal in both cases is to filter out (random) noise so we can build a portfolio based on (non-random) signal; in other words, we want to refine the information content. In the next lesson, we delve into information theory and learn methods for quantifying and reducing uncertainty and evaluating information directly.

References

- Akinshin, Andrey. "The Importance of Kernel Density Estimation Bandwidth." *Aakinshin*, 13 Oct. 2020, <https://aakinshin.net/posts/kde-bw/>
- de Prado, Marcos López. *Machine Learning for Asset Managers*. Cambridge University Press, 2020.
- Moen, Endre. "Machine_Learning_for_Asset_Managers." *GitHub*, https://github.com/emoen/Machine-Learning-for-Asset-Managers/tree/master/Machine_Learning_for_Asset_Managers.
- Scikit-learn developers. "2.8.2. Kernel Density Estimation." *Scikit-learn*, 2011, <https://scikit-learn.org/stable/modules/density.html#kernel-density>.
- Timm, Bjarne. "Utilizing Machine Learning to Address Noise in Covariance and Correlation Matrices: An Application and Modification of Enhanced Portfolio Optimisation." 2021, Copenhagen Business School, Master's thesis, <https://research.cbs.dk/en/studentProjects/utilizing-machine-learning-to-address-noise-in-covariance-and-cor>.
- Pedersen, Lasse Heje, Abhilash Babu, and Ari Levine. "Enhanced Portfolio Optimization." *Financial Analysts Journal*, vol. 77, no. 2, 2021, pp. 1–49. <https://doi.org/10.1080/0015198X.2020.1854543>.
- Vanderplas, Jake. "05.13-Kernel-Density-Estimation.ipynb." *GitHub*, <https://github.com/jakevdp/PythonDataScienceHandbook/blob/master/notebooks/05.13-Kernel-Density-Estimation.ipynb>.
- Zhang, Wensheng, Cheng-Xiang Wang, Xiaofeng Tao, and Piya Patcharamaneepakorn. "Exact Distributions of Finite Random Matrices and Their Applications to Spectrum Sensing." *Sensors*, vol. 16, no. 8, 2016, <https://doi.org/10.3390/s16081183>.